

Тема 5. Концептуален модел и поглед „отгоре“. Use case диаграми. Актьори и зависимости. Sitemap

1.	Концептуален модел	3
А)	Какво представлява?	3
Б)	Пример.....	3
<i>Google Maps</i>		3
В)	Обекти на концептуалния модел (Entities)	5
Г)	Класове на обектите и инстанции (Entity Classes & Instances)	6
Д)	Връзки (Conceptual Model Relationships).....	6
Е)	Ограничения (Conceptual Model Constraints)	7
☐	One and only one	7
☐	Zero or one.....	7
☐	More than one	7
☐	One or more.....	7
☐	Zero or more	7
Ж)	Пример.....	7
2.	Поглед „отгоре“	8
Плюсове:		9
Минуси:		9
3.	Use Case диаграми	9
А)	Какво представлява?	9
1)	Актьорът. Потребителят на системата	10
2)	Целта.	10
3)	Системата.....	10
Б)	Характеристики	10
☐	Организира функционалните изисквания	10
☐	Моделира целите на взаимодействията между системата и актьорите	10
☐	Записва пътищата (сценариите) от стартиращите събития до целите	10
☐	Описва основния поток на събития и различни алтернативни потоци	11
☐	Е многостепенен, така че един Use Case може да използва функционалността на друг	11
В)	Как да напишете Use Case	11
Г)	Предимства на Use Case.....	12
4.	Актьори и зависимости	14

A) Актьор	14
☐ Потребител	14
☐ Външна система.....	14
☐ Автоматизиран процес	14
☐ Администратор.....	14
Б) Зависимости	15
☐ Include (Включва).....	15
☐ Extend (Разширява).....	15
☐ Generalization (Обобщава)	15
☐ Dependency (Зависи)	16
В) Пример.....	16
5. Sitemap	17
A) Какво представлява Sitemap?	17
Б) Защо се използва?.....	17
В) Как се създава Sitemap?.....	18
1) Събиране на съдържание	18
2) Обмисляне как ще се използва сайтът	19
3) Определете основните си страници	19
4) Добавете вторичните си страници.....	19
5) Свържете всички страници	19
6) Получете обратна връзка.....	19
Г) Пример	20

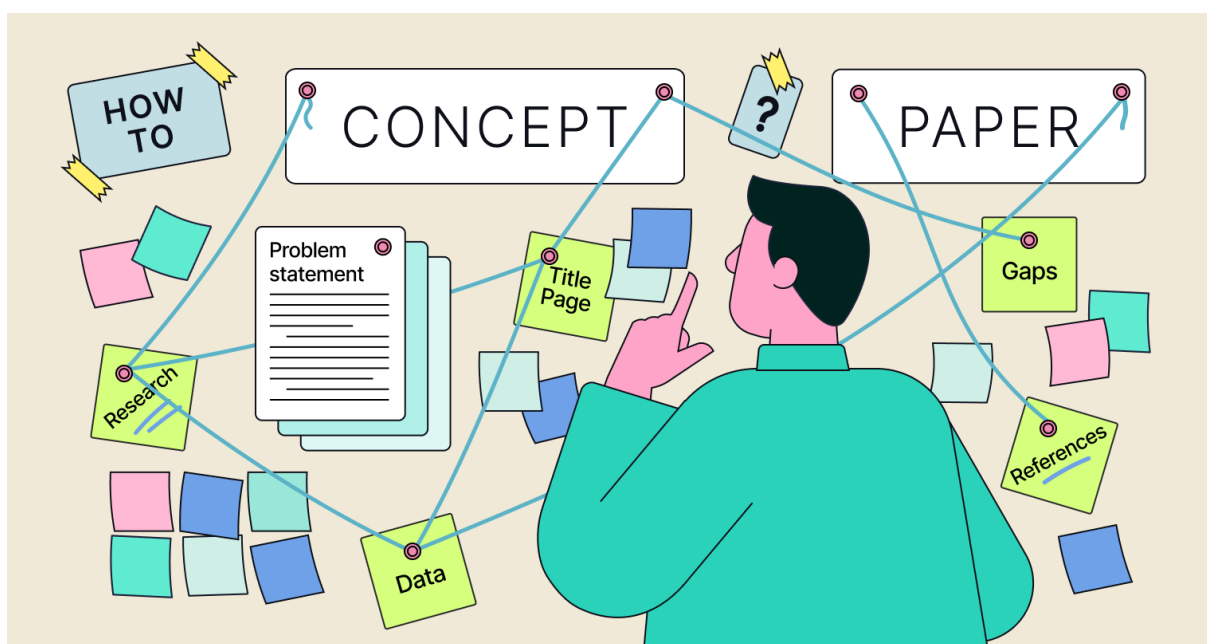
1. Концептуален модел

А) Какво представлява?

Когато проектираме потребителски интерфейс и потребителски опит, ние започваме с концепция. Имаме идея за това как трябва да функционира системата в реалния свят. Просто трябва да я споделим на останалите членове на екипа.

Концептуалният модел е представяне на система. Той показва как хора, места и неща взаимодействат. Те показват реалния свят и взаимодействията на нашата идея за дизайн.

Изразено по-просто, концептуалният модел е абстракция на част от реалния свят или на дизайн, който планираме да внесем в реалността.

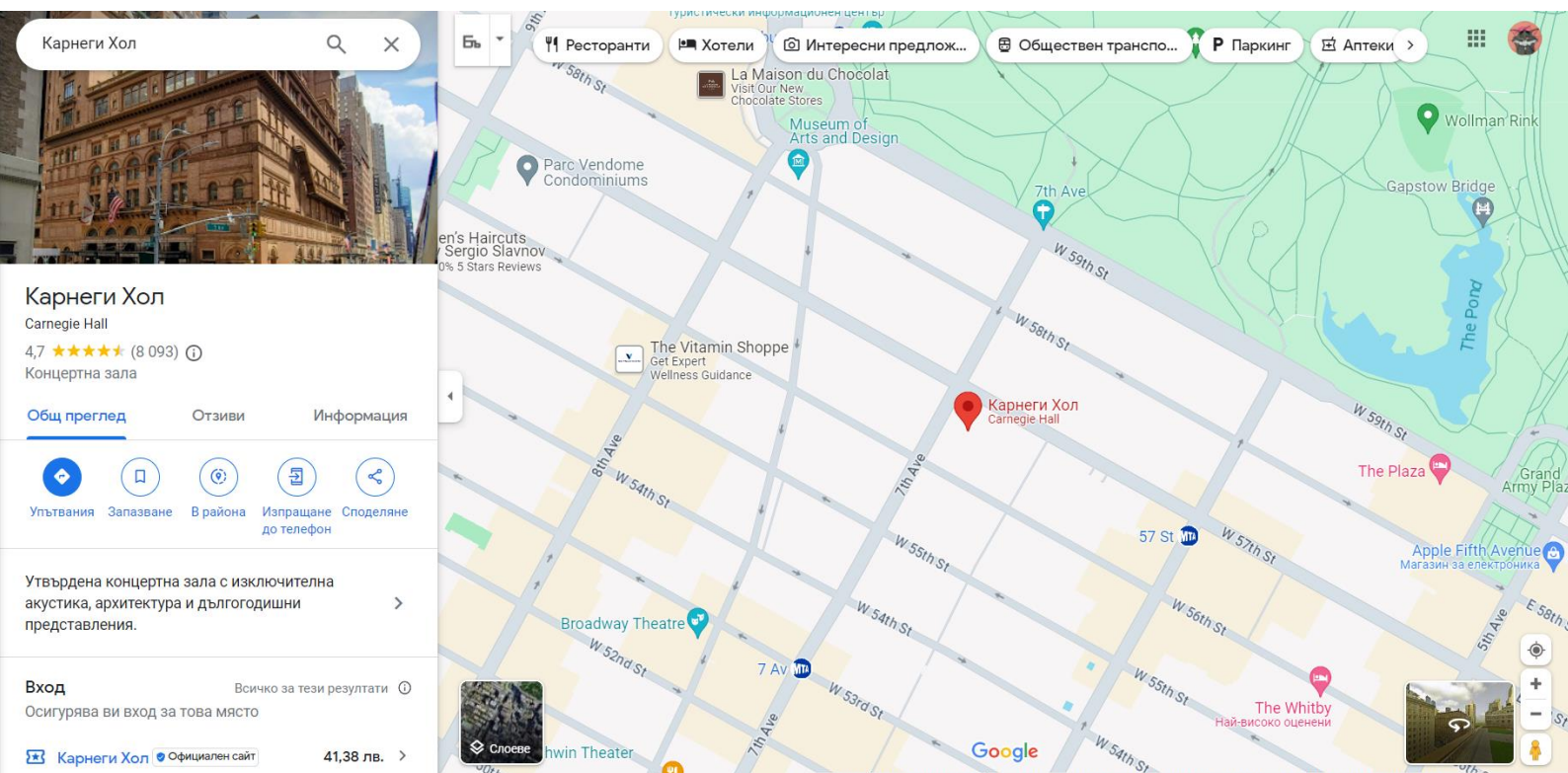


Б) Пример

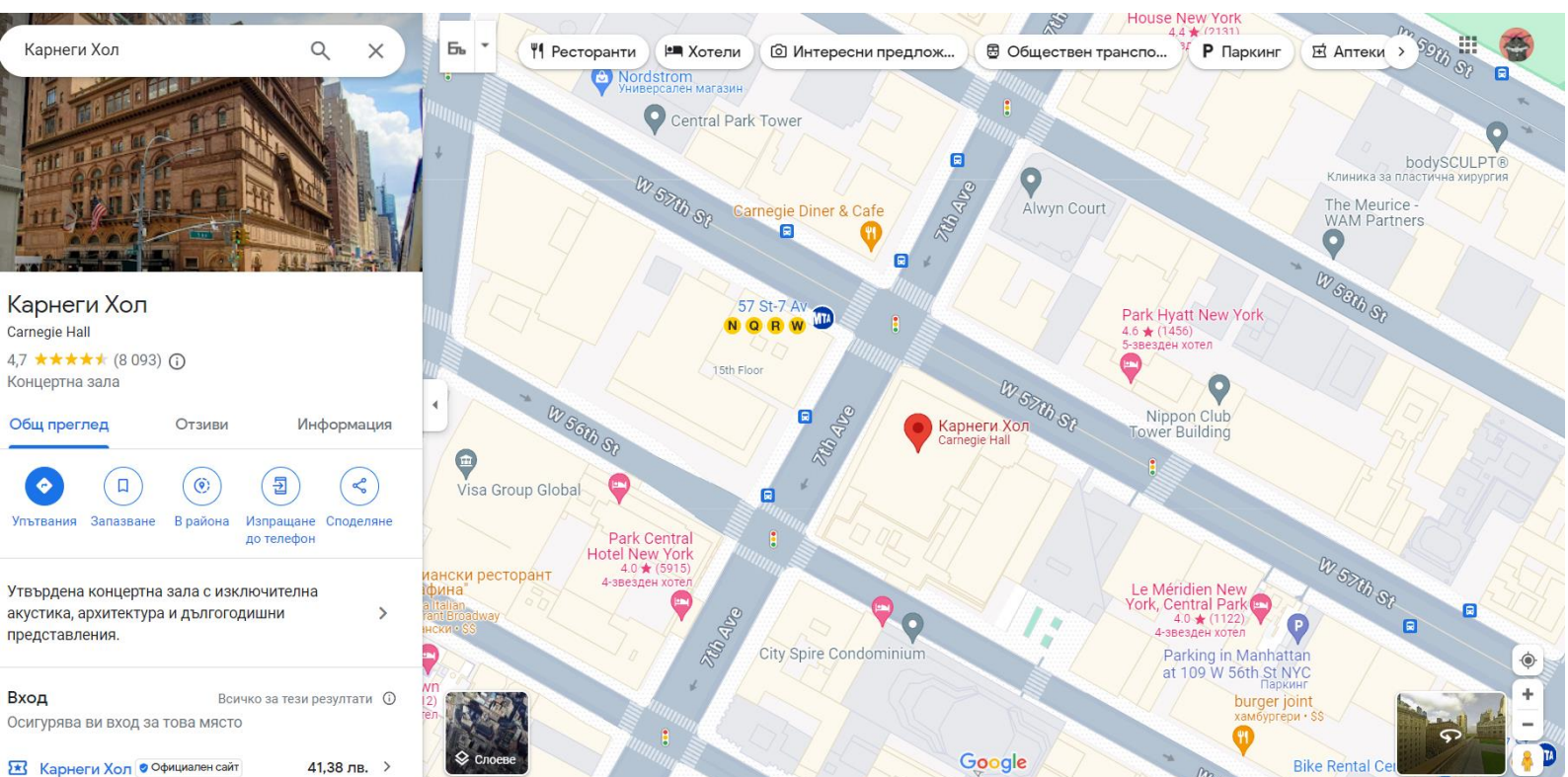
Google Maps - Когато отворите Google Maps на вашия мобилен телефон или друго устройство, ще видите представяне на пътища, мостове, кръстовища, ресторанти, магазини и забележителности.

Нека кажем, че искате да посетите Карнеги Хол (концертна зала) в Ню Йорк.

Въвеждате "Карнеги Хол" в търсачката. Това ви довежда до карта на района около 7-мо Авеню и Уест 57-ма улица в Ню Йорк, Ню Йорк, САЩ.



Ако мащабираме ще видим, че има ресторанти, закусвални и хотели наблизо. Тези обекти се появяват на картата като скици на сгради с маркери, които показват техните местоположения.



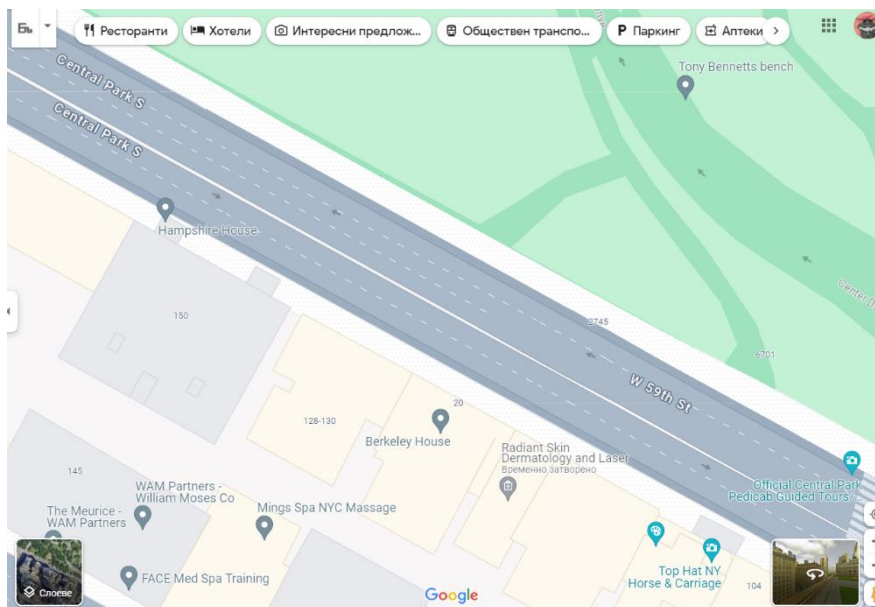
Картографите в Google Maps ви показват концепцията на района в Ню Йорк близо до Карнеги Хол.

Проектиране на потребителски интерфейс

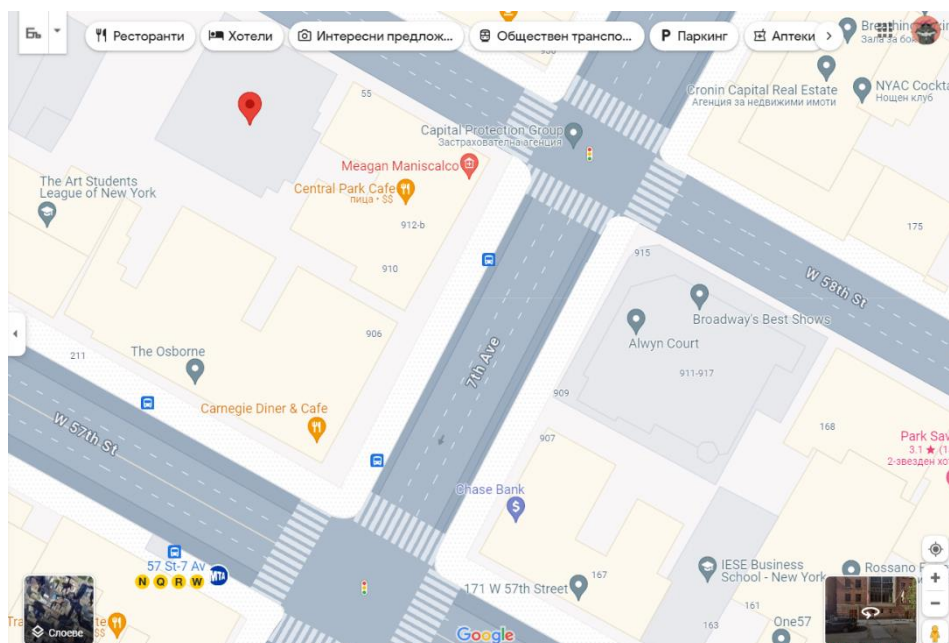
В) Обекти на концептуалния модел (Entities)

Обектите на концептуалните модели представляват обекти от реалния свят в една система. Нека отново вземем Google Maps за пример.

Ще забележите, че има линии, които представят улиците. Две паралелни линии представляват Уест 59-та улица, за да покажат, че тя е двупосочна улица.



7-мо Авеню с една линия, защото е еднопосочна улица. Тя има стрелки, които показват посоката на движение на трафика в реалния свят.



Г) Класове на обектите и инстанции (Entity Classes & Instances)

Класовете на обектите (Entity Classes) в концептуалния модел представляват обекти в дадена система, които са от значение за моделирането на бизнес процесите и данните. Тези класове обикновено се използват в концептуалното моделиране, за да се идентифицират и организират същностите, които са съществени за дадената система или дейност. Класовете на обекти са универсални.

Инстанцията на клас (Instance) представлява конкретно обектно представяне на даден клас.

Например:

Клас : Улица

Уест 57-ма улица

7-ма Авеню

Бродуей

Клас - Ресторант

Chai Thai Kitchen

Бруклин Дайнър

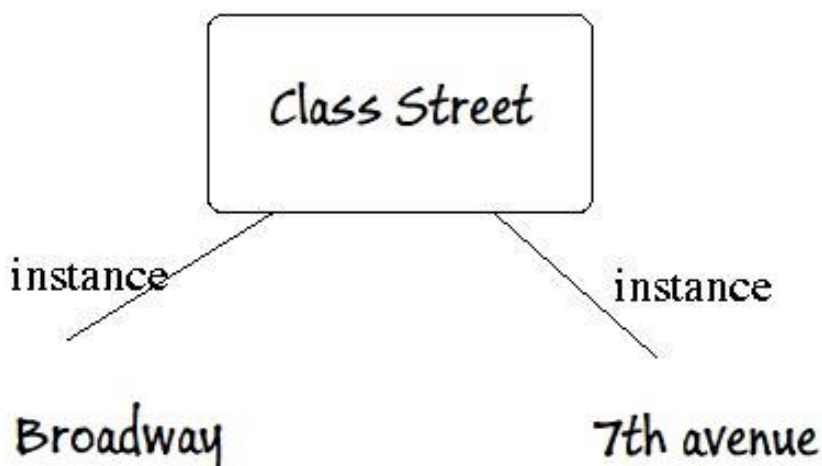
P.J. Карни Пъб

Клас - Хотел

Парк Централ Хотел

Уелингтън

Courtyard by Marriott New York



Д) Връзки (Conceptual Model Relationships)

Взаимоотношенията в концептуалния модел изобразяват взаимодействие между обектите.

Това е като съставяне на изречение, където имате подлог, сказуемо и допълнение. Взаимоотношението е сказуемото. Обектите са подлогът и допълнението.

Например:

- Карън Смит (обект) прави регистрация (взаимоотношение) в хотел Парк Централ (обект).
- Билетната каса (обект) продава (взаимоотношение) билети (обект).

Е) Ограничения (Conceptual Model Constraints)

Ограниченията управляват връзките между обектите в концептуален модел. Една инстанция може да взаимодейства с цял клас. Клас може да взаимодейства с други класове. Или инстанция може да взаимодейства само с една друга инстанция. Клас може и да не взаимодейства с други класове. Някои възможни ограничения са:

- One and only one - Това показва, че в системата има само един обект от този тип. Например, оркестърът има един диригент.
- Zero or one - Можете да го разглеждате като ограничение "да" или "не" или "включено" или "изключено". Например, един клиент, който посещава представление в Карнеги хол, има сезонен абонамент. Другият няма. Обектът, който наричаме "сезонен абонамент", има стойност на ограничението 1 във връзката си с първия клиент. Със втория клиент стойността е 0.
- More than one - Това се изразява като минимална и максимална стойност. Оркестърът може да има нужда от поне 4 цигулари. Пълен симфоничен оркестър има най-много 34. Стойността на ограничението за обекта, наречен "Цигулар", е от 4 до 34 във връзка с обекта, наречен "Оркестър".
- One or more - Тук трябва да имате поне 1 обект. Обект с име "Музиканти" трябва да има стойност от поне 1, за да има връзка с обекта "Музикално изпълнение".
- Zero or more - Това казва, че е възможно да няма такъв вид обект и че можете да имате един или повече. Може да не продадете нито един билет за представление. Но ако залата за представления има 3,671 места, това е максималната стойност.

Ж) Пример

Обекти - хората, местата и нещата.

Взаимоотношения - как обектите взаимодействат помежду си.

Ограничения - броят на взаимодействащите обекти.

Нека представим, че създавате концептуален модел за база данни за Карнеги хол. Някои от ограниченията могат да изглеждат така:

Хора

Музиканти

Диригенти - 1 и само 1

Цигулари - от 4 до 34

Виоли - от 4 до 12

Клиенти

Притежатели на сезонен абонамент - от 0 до 1 000

Робърт Джонсън - 1 и само 1

Представления

Събитие на 5 април 2021 г.

Продадени билети - от 0 до 3 671

Непродадени билети - от 0 до 3 671

И връзките между тях могат да бъдат такива:

Диригенти (1 и само 1) към цигулари (от 4 до 34).

Клиенти (от 0 до безкрайност) към места (от 0 до 3 671).

Робърт Джонсън (1 и само 1) към сезонни абонаменти (0 или 1).

2. Поглед „отгоре“

Погледът "отгоре" предполага анализ на продукта или услугата, като започнете от общата си представа за него и постепенно се движите към по-детайлни аспекти. Този подход е особено полезен при изследване на производителността на продукта или при определяне на тенденции и модели в употребата му от потребителите. Той ви позволява да придобиете цялостно разбиране за начина, по който продуктът се използва и как може да бъде подобрен, като се отчитат общите тенденции и нуждите на потребителите.



Този подход е особено важен и за анализ на потребителския интерфейс (UI) и потребителския опит (UX). Започването с поглед "отгоре" ви позволява да разгледате общите тенденции и модели в потребителското поведение и използването на продукта или услугата.

Плюсове:

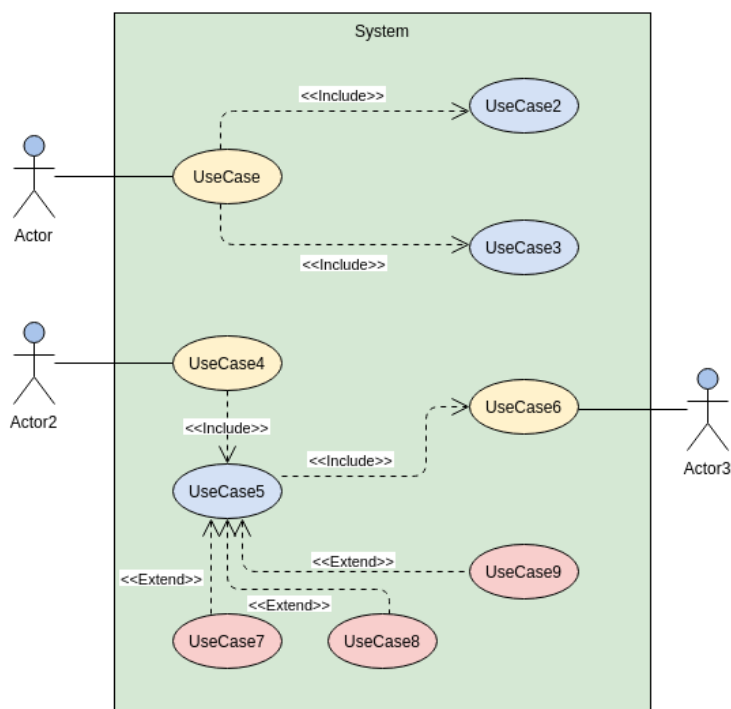
- Предоставя общо представление за използването на продукта
- Помага за идентифициране на общи тенденции и модели в употребата
- Може да бъде полезен за идентифициране на области на продукта, които се използват най-много или най-малко

Минуси:

- Може да пропусне важни детайли, които са видими само на компонентно ниво
- Може да пренебрегне вариациите в моделите на употреба между различни групи потребители или контексти
- Може да е по-малко ефективен за идентифициране на конкретни проблеми или проблеми с продукта

3. Use Case диаграми

А) Какво представлява?



Use Case е методология, използвана в системния анализ за идентифициране, разясняване и организиране на изискванията на системата. Use Case се състои от набор от възможни последователности на взаимодействия между системите и потребителите в определена среда и свързани с определена цел. Методът създава документ, който описва всички стъпки, които потребителите предприемат, за да завършат дадена дейност.

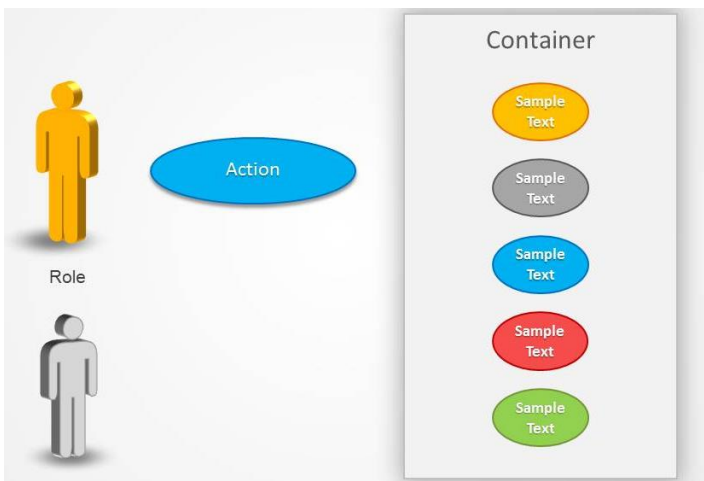
Бизнес анализаторите обикновено са отговорни за писането на Use Case и те се използват по време на няколко етапа от

разработката на софтуера, като планиране на системните изисквания, валидиране на

дизайна, тестване на софтуера и създаване на обща структура за онлайн помощ и потребителски ръководства.

Всеки Use Case съдържа три основни елемента:

- 1) **Актьорът. Потребителят на системата** - това може да бъде единично лице или група хора, които взаимодействат с процеса.
- 2) **Целта** - Крайният успешен резултат, който завършва процеса.
- 3) **Системата** - Процесът и стъпките, които се предприемат, за да се постигне крайната цел, включително необходимите функционални изисквания и техните предвидени поведения.



Б) Характеристики

- **Организира функционалните изисквания** - Use Case предоставя структуриран начин за организиране и документиране на функционалните изисквания на системата. Те помагат за разбиране на това, какво системата трябва да прави от гледна точка на крайните потребители.
- **Моделира целите на взаимодействията между системата и актьорите** - Use Case се фокусира върху улавянето на целите, които актьорите (потребителите или външните системи) искат да постигнат, като взаимодействат с системата. Те описват предвидените резултати от тези взаимодействия.
- **Записва пътищата (сценариите) от стартиращите събития до целите** - Use Case описва последователността от стъпки или събития, които се случват от първоначалното стартиращо събитие (като действие на потребител) до постигането на целите.

- Описва основния поток на събития и различни алтернативни потоци - Use Case определя основния поток на събития и различните алтернативни потоци, които могат да се случат при различни условия или вариации в процеса.
- Е многостепенен, така че един Use Case може да използва функционалността на друг - Use Case може да бъде многостепенен, така че един случай да използва функционалността на друг. Това позволява на анализаторите да създадат по-голяма структура и да използват повторно функционалността при необходимост.

В) Как да напишете Use Case

Когато пишете Use Case, вие трябва да обмислите обхвата на дизайна, за да идентифицирате всички елементи, които лежат в границите на процесите и извън тях. Всичко съществено за Use Case, което лежи извън границите му, трябва да бъде отбелязано с подкрепящ актьор или чрез друг Use Case. Обхватът на дизайна може да бъде конкретна система, подсистема или фирма. Use Case, които описват бизнес процеси, обикновено са на обхват на фирмата.

Както бе споменато, трите основни елемента, които образуват Use Case са актьори, системата и целта. Други допълнителни елементи, които трябва да се вземат предвид при писането на Use Case включват:

- Заинтересованите страни, или всеки, който има интерес или инвестиция в начина, по който системата изпълнява.
- Предпоставки, или елементите, които трябва да са верни, преди да може да се случи Use Case.
- Тригери, или събитията, които причиняват началото на Use Case.
- Пост-условия, или какво системата трябва да е завършила към края на стъпките.

Диаграмата на Use Case се създава, използвайки единен модел на езика, като за всяка стъпка се представя името ѝ в овал; всеки актьор се представя със символ на човек и неговото име отдолу; всяко действие се показва с линия между актьора и стъпката; а границите на системата се показват с правоъгълник около Use Case.

Процесът на писане включва:

1. Идентифициране на всички потребители на системата и създаване на профил за всеки един. Това включва всяка роля, изпълняван от потребител, който взаимодейства със системата.
2. Избор на един потребител и дефиниране на неговата цел - или това, което потребителят се надява да постигне, като взаимодейства със системата. Всяка от тези цели става употреба на случай.
3. Описание на пътя, взет за всеки случай на употреба през системата, за да се постигне тази цел.
4. Разглеждане на всеки алтернативен ход на събитията и разширяване на употребата на случаи - или различните пътища, които могат да бъдат избрани, за да се постигне целта.
5. Идентифициране на общи черти в пътуванията, за да се създадат общи употреби на случаи и да се напишат описания за всеки един.
6. Повтаряне на стъпки две до пет за всички други потребители на системата.

Use case methodology



Г) Предимства на Use Case

Единичният Use Case може да бъде от полза за разработчиците, като разкрива как трябва да се поведе системата, като същевременно помага да се идентифицират възможни грешки, които могат да възникнат по време на процеса.

Другите ползи от разработката на Use Case включват:

- Създаването на списък с цели в процеса на писане на Use Case, които могат да се използват за определяне на сложността и разходите на системата.

- Чрез фокусиране и върху потребителя, и върху системата, реалните нужди на системата могат да бъдат идентифицирани по-рано в процеса на дизайн.



- Use Case са лесно разбираеми от заинтересованите страни, включително клиенти, потребители и изпълнителни директори - не само от разработчици и тестери.

- Създаването на разширяващи Use Case и идентифицирането на изключения от успешните сценарии на Use Case спестява време на разработчиците, като прави по-лесно дефинирането на тънките изисквания на системата.

- Чрез идентифицирането на границите на системата в обхвата на дизайна на Use Case, разработчиците могат да избегнат надхвърляне на обхвата.

- Може да се избегне преждевременният дизайн, като се фокусира върху това, което системата трябва да прави, вместо как да го направи.

Освен това, Use Case могат лесно да бъдат превърнати в тестови случаи, като се съпоставят общите и алтернативни пътища и се събира тестова информация за всеки от сценариите. Тези функционални тестови случаи ще помогнат на екипа за разработка да гарантира, че всички функционални изисквания на системата са включени в плана за тестове.

Освен това, Use Case могат да бъдат използвани и в различни други области на разработката на софтуер, включително планиране на проекти, потребителска документация и дефиниции на тестови случаи. Use Case могат също да бъдат използвани като инструмент за планиране за итеративно развитие.

4. Актьори и зависимости

А) Актьор

Use Case актьорът представлява всеки обект (човек, друга система или външен агент), който взаимодейства със системата, която се моделира. Той е важен участник във всеки сценарий на използване (Use Case), който се дефинира в системата.



В контекста на Use Case диаграмите, актьорите се изобразяват като външни елементи, обикновено като човешка икона или познати символи за представяне на различни видове външни агенти. Те не са част от системата, но влияят на нея чрез своите входове (inputs) и изходи (outputs).

Примери за актьори в Use Case диаграмите могат да бъдат:

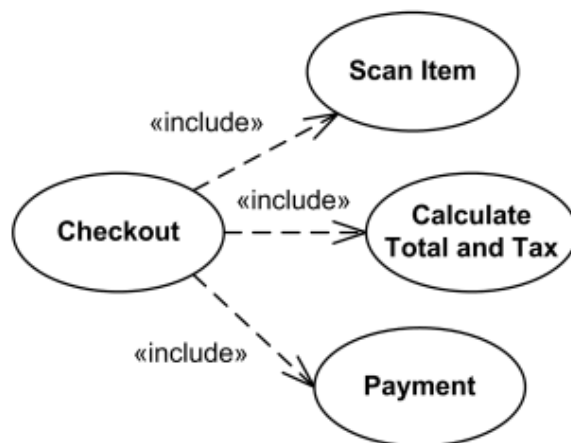
- **Потребител** - Човек, който използва системата за изпълнение на различни функционалности, като въвеждане на данни, наблюдение на информация, извършване на действия и други.
- **Външна система** - Друга компютърна система или софтуер, която комуникира със системата, като изпраща и получава данни, извиква определени услуги или използва функционалност.
- **Автоматизиран процес** - Процес или механизъм, който е интегриран в системата и извършва определени действия автоматично без непосредствено участие на човек.
- **Администратор** - Човек или роля, която управлява и поддържа системата, като извършва конфигурации, добавя и премахва потребители, управлява права за достъп и други административни задачи.

В общия случай, актьорите се идентифицират чрез техните роли и отговорности в системата и са ключови за определянето на Use Case сценариите, които трябва да бъдат моделирани.

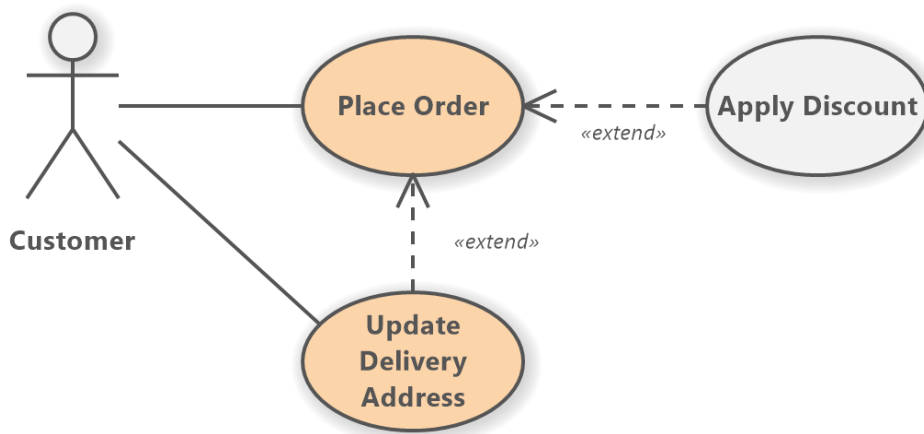
Б) Зависимости

В една Use Case диаграма може да има няколко вида зависимости между различните елементи. Ето някои от тях:

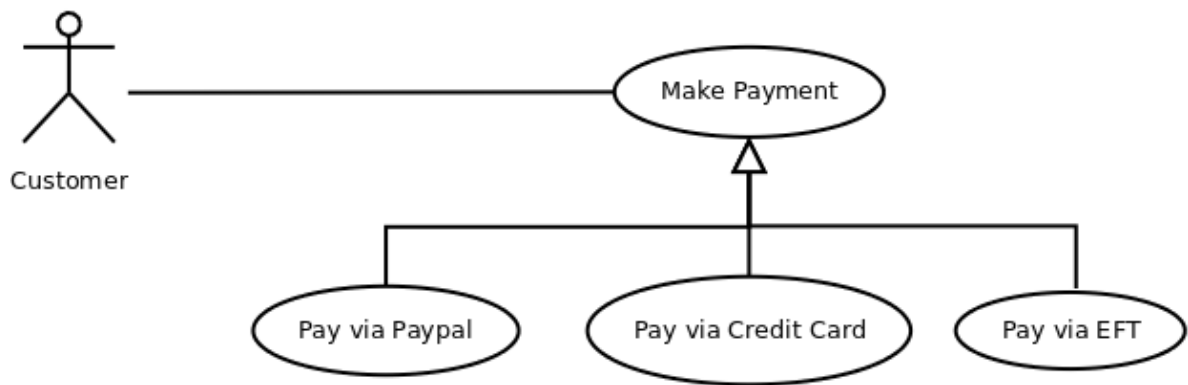
- **Include (Включва)** - Един случай на използване може да включва друг. Например, ако имате "Извършване на плащане", той може да включва "Въвеждане на информация за плащане".



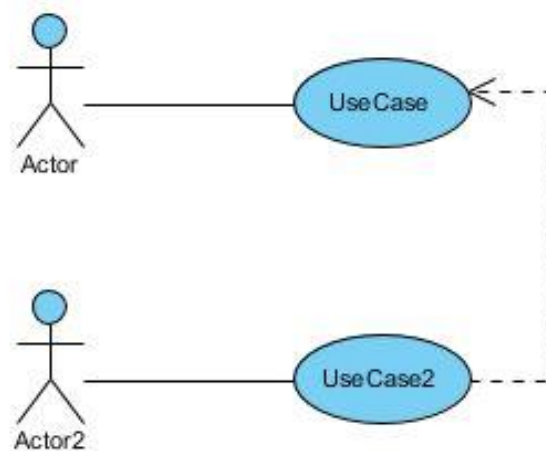
- **Extend (Разширява)** - Един случай на използване може да разшири друг. Това обикновено се използва, когато има опционални функционалности. Например, ако имате случай на използване "Изпращане на съобщение", той може да разшири случая на използване "Добавяне на прикачен файл".



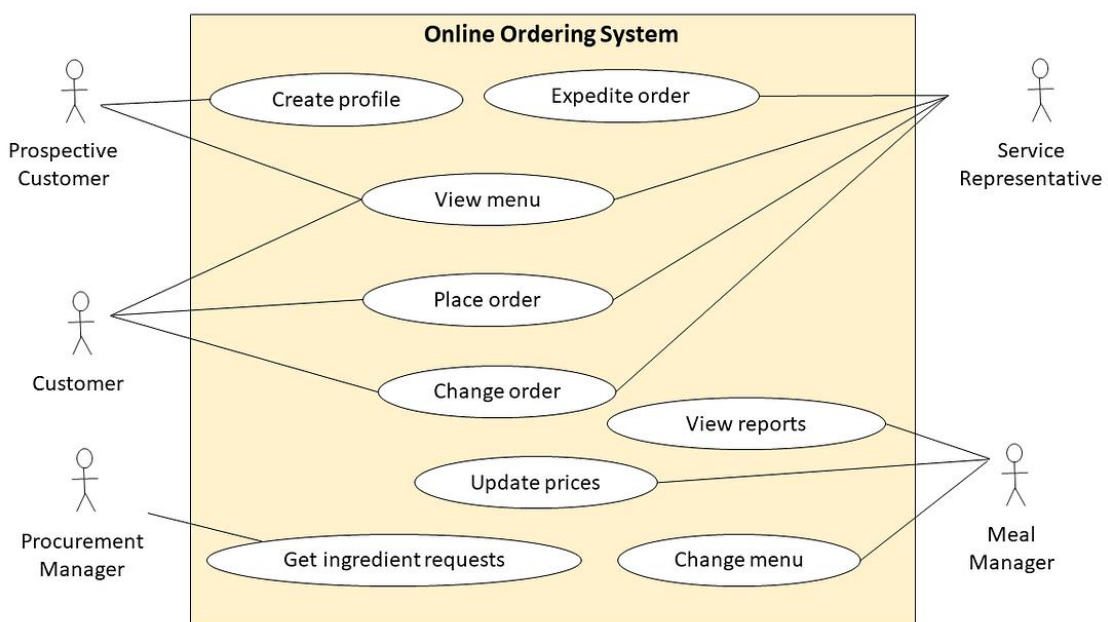
- **Generalization (Обобщава)** - Един случай на използване може да бъде обобщение на друг. Това се използва, когато един случай на използване е по-общ и включва функционалността на други случаи на използване. Например, ако имате случай на използване "Извършване на плащане с карта" и "Извършване на плащане с PayPal", те могат да бъдат обобщени в по-общ случай на използване "Извършване на плащане".



- **Dependency (Зависи)** - Един елемент в диаграмата може да зависи от друг елемент. Например, ако имате случай на използване, който изисква определена услуга от друга система, тогава има зависимост между двата елемента.



В) Пример



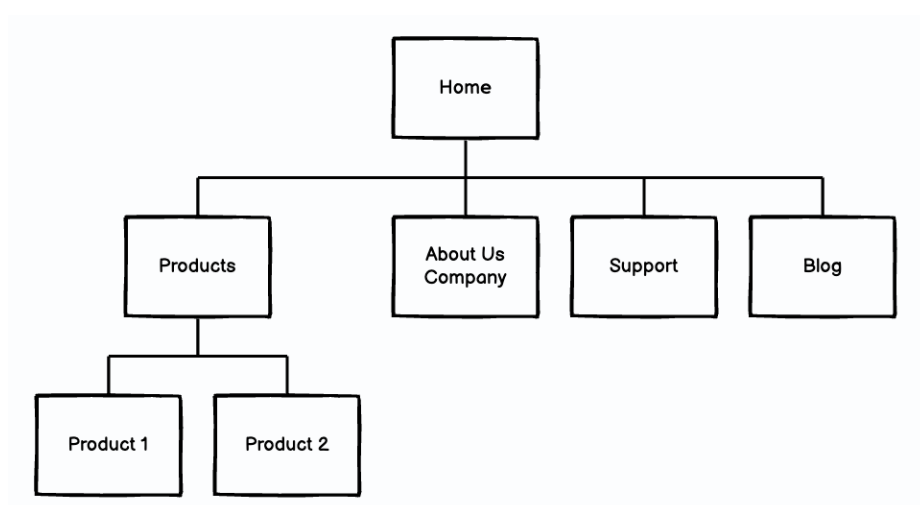
5. Sitemap

А) Какво представлява Sitemap?

Sitemap е визуално представяне на структурата на уебсайт, което показва различните страници и тяхната йерархия. Това е като карта или схема, която помага на потребителите да се ориентират в продукта и да навигират между различните страници.

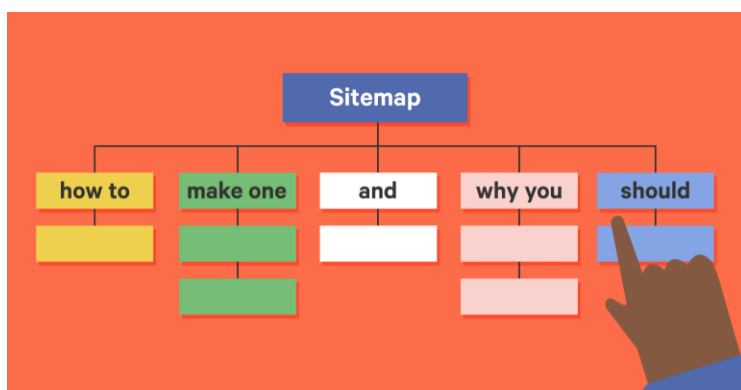
Целта на картата е да ви позволи да визуализирате как отделните уеб страници или раздели на сайта са свързани помежду си. Те също ви помагат да разберете как потребителите навигират из уебсайта.

Картата на уебсайта е основно ръководство, което описва основните страници на сайта и техните взаимоотношения, показвайки как са свързани и къде всяка страница се вписва в по-голямата картина.



Б) Защо се използва?

Sitemap ви помагат да следите какво правите, къде отивате и как уебсайтът ще се развива с течение на времето. Те помагат за комуникацията между екипите, участващи в разработката на сайта. Също така те идентифицират всякакви области, които могат да бъдат пропуснати в ранен етап, предоставяйки допълнително разбиращелство за проблеми, които може да не са очевидни, когато се разглежда индивидуалната структура на страниците.

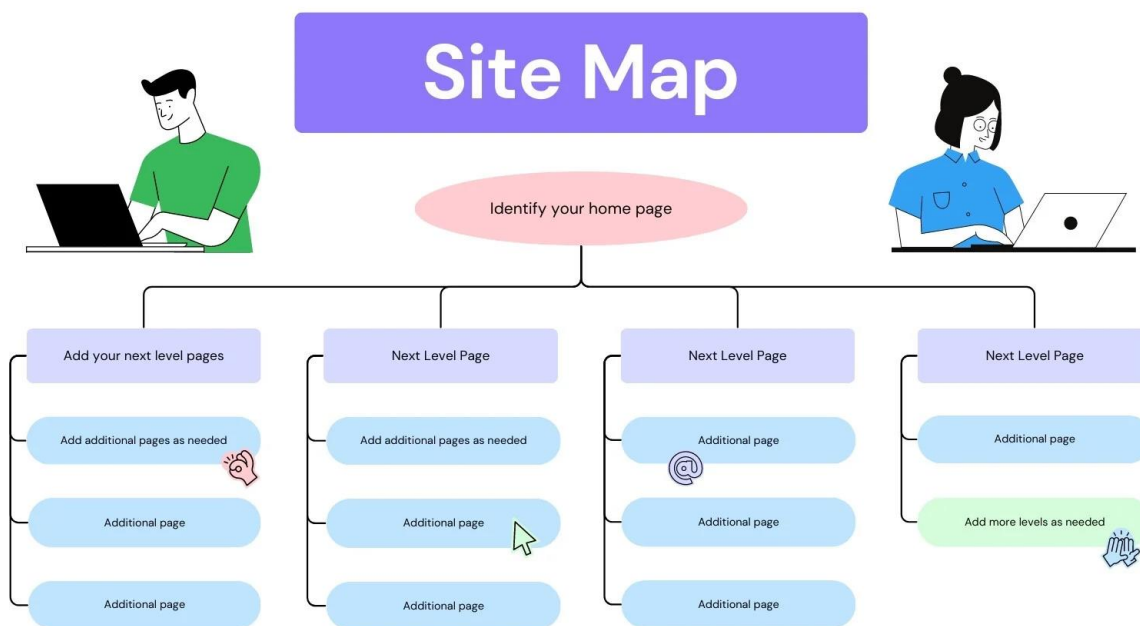


Sitemap е полезен инструмент за получаване на одобрение от заинтересованите страни. Създаването на картата може да изглежда като ненужна стъпка, но, когато става дума за това, това е за правилната структура на вашия сайт. Този документ е доказателство, че сте помислили за всички пътища за потребителското пътуване по вашия сайт. Той показва, че мислите за това кой използва вашия уебсайт и техните нужди.

В) Как се създава Sitemap?

Има шест основни етапа за създаване на Sitemap:

1. Организирайте съдържанието си
2. Мислете за начина, по който ще се използва вашият сайт
3. Определете основните си страници
4. Добавете вторичните си страници
5. Свържете страниците си
6. Споделете с по-широкия екип



Нека подробно разгледаме всеки от тези етапи.

1) Събиране на съдържание

За да създадете карта на UX, първото, което трябва да направите, е да подредите цялото съдържание. Това може да стане, като създадете таблица, в която да изброите всеки един раздел на вашия сайт и неговото текущо заглавие на страницата.

След като сте готови с това, преминете през всеки раздел и решете кои страници са достатъчно важни, за да бъдат включени в самата карта на сайта. Също така е добра практика

да групирате подобни страници в папки или подменюта. Например, ако имате пет варианта за "Свържете се с нас" с различни пътища към тях, разгледайте възможността те да бъдат сортирани под едно меню, вместо под пет.

След като организирате всичко, е време да започнете да мислите за вашите потребители и да създадете вашата карта.

2) Обмисляне как ще се използва сайтът

Първата стъпка за създаване на карта на сайта е да се уверите, че тя съответства на вашите дизайн цели. Трябва да отговорите на следните въпроси:

- Какви проблеми се опитвате да решите?
- Какво искате да подобрите?

Можете да използвате клиентски персони или да създадете персона на базата на текущите потребители, с които проведохте проучване. Трябва да разберете кои са вашите потребители и какво искат (иначе няма да знаете как да решите техните проблеми).

3) Определете основните си страници

Започнете да изброявате основните страници на вашия уебсайт и ги направете приоритетни. "За нас", "Продукти", "Блог" и "Свържете се с нас" са примери за страници, които обикновено заемат важно място. Придавайте им приоритет във важността за вашите потребители.

4) Добавете вторичните си страници

Започнете да изброявате вторичните си страници. Те ще бъдат неща, които искате да включите, но няма да бъдат толкова важни, колкото основните ви страници. Помнете, вторичните страници трябва да бъдат вложени под основните. Те са все още важни за наличие, но няма да привличат вниманието на потребителите толкова много, колкото основните страници.

5) Свържете всички страници

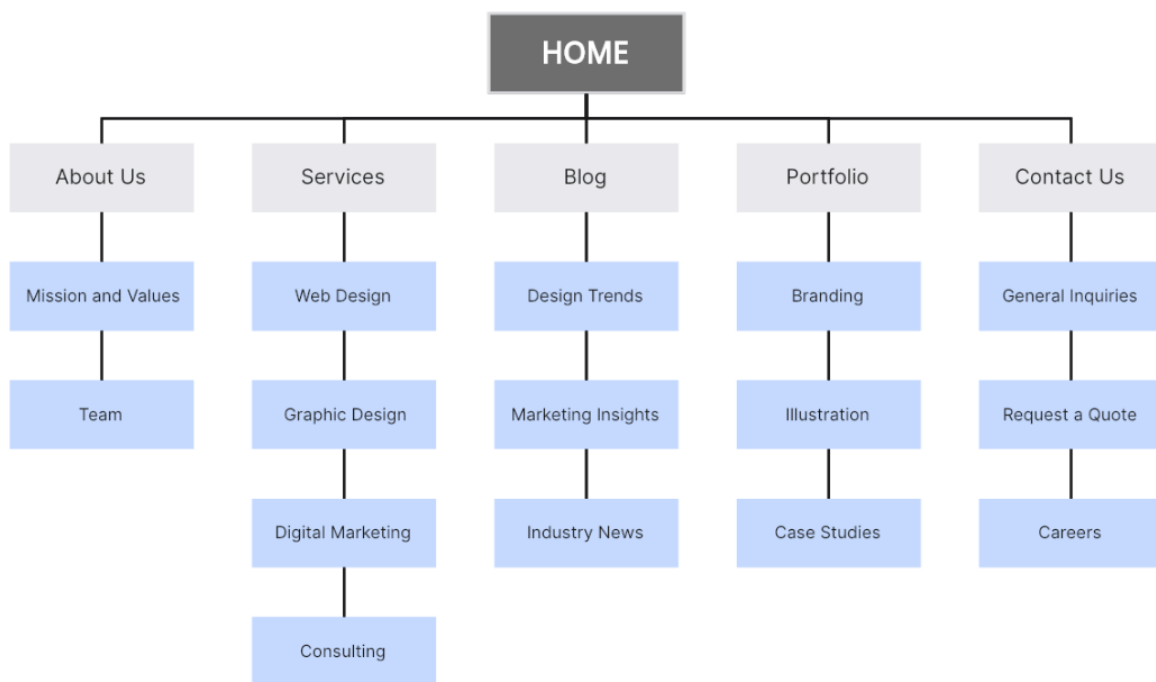
Организирайте вашите приоритетни страници и поставете вторичните страници под тях, така че да получите структура, подобна на дърво. По този начин можете да видите всичките си страници и става лесно да добавяте нови страници, ако е необходимо.

6) Получете обратна връзка

Може да искате да получите обратна връзка от вашите заинтересовани страни. Целта тук е всички да се съгласят кои страници са най-важни, така че да можете да оформите вашите основни страници, докато имате малко число. След като сте определили основните страници, преминете през същия процес с вторичните и третичните страници.

Сега вече трябва да имате ясна представа за начина, по който сайтът ви е структуриран и организиран.

Г) Пример



И Use Case диаграмата и Sitemap могат да бъдат реализирани на miro.com.